

1. Introducción

La temática que elegí es una **academia de magia**: los 23 personajes son brujas y magos de la Academia Umbraluz, y la interfaz está en violeta y dorado. Los atributos del enunciado encajan sin forzar nada: el género distingue brujas de magos y la calvicie separa a los profesores de los aprendices.

Las estrategias que usé son estas. **Divide y Conquista** aparece en tres lugares: MergeSort para el lote inicial de 23 personajes, búsqueda binaria para ubicar un alta individual posterior, y la reducción del conjunto de candidatos turno a turno, que es el corazón del juego. **Greedy** lo uso para la selección del filtro, con criterio minimax. Además uso **acceso por clave O(1)** con HashSet (filtros ya preguntados, verificación de duplicados) y HashMap (del id del personaje a su tarjeta).

Dos aclaraciones, porque no todo está usado igual: **burbujeo e inserción simple están implementados pero no se usan en el juego**, solo alimentan la tabla de la sección 5; y además de la greedy programé una **segunda máquina de búsqueda secuencial**, que juega peor a propósito, para poder comparar las dos estrategias con números.

2. Diagrama de clases

El diagrama está en la última hoja, partido en dos vistas: el **núcleo algorítmico** (paquetes modelo, catalogo y jugador) y el **motor y las vistas** (paquetes juego, app y vista). Lo dividí así porque en un solo dibujo con las veintipico de clases juntas no se leía nada.

2.1 Relaciones

Herencia. Dos jerarquías: Filtro (abstracta) con sus cuatro subclases, y Jugador (abstracta) con JugadorHumano y JugadorMaquina, que a su vez tiene MaquinaGreedy y MaquinaSecuencial. La de Filtro permite que el greedy recorra todos los filtros con **un solo bucle**, sin un if por característica: le alcanza con llamar a evaluar(personaje). La de Jugador está pensada así porque **el tablero del humano se achica con el mismo algoritmo que el de la máquina**: todo el D&C vive en la clase padre, y lo que agrega JugadorMaquina es decidir sola qué jugar y la función de factibilidad.

Realización de interfaces. OraculoPersonaje implementa Oraculo; tres clases implementan RegistroRazonamiento; VentanaJuego implementa Observador. Jugada es una sealed interface con dos implementaciones: al ser sellada, el compilador garantiza que nadie invente una tercera y que los switch estén completos.

Composición. Partida contiene dos OraculoPersonaje, uno por jugador; CatalogoPersonajes los 23 Personaje; EvaluacionFiltro un Filtro y una Particion.

Dependencia. CatalogoPersonajes usa OrdenadorPersonajes y VerificadorCatalogo, MaquinaGreedy usa EvaluacionFiltro, y las clases de app dependen del motor, pero el motor no las conoce.

2.2 Justificación del modelo de datos

Estructura	Dónde	Por qué esa y no otra
ArrayList<Personaje>	candidatos, catálogo	El orden importa y necesito acceso por índice en O(1). La binaria hace get(medio): con ArrayList cuesta O(1), con LinkedList costaría O(n) y la búsqueda binaria perdería todo el sentido , quedando peor que una lineal.
HashSet<String>	filtrosUsados	Solo me importa saber si un filtro ya se preguntó: es pertenencia, no orden. contains() en O(1) contra O(n) de una lista.
HashSet<String>	verificarUnicidad()	add() devuelve false si el elemento ya estaba, así que detecto duplicados en una pasada, $\Theta(n)$.
HashMap<Integer, JLabel>	tarjetas del tablero	Comparando todos contra todos sería $\Theta(n^2)$. Necesito ir del id del personaje a su tarjeta para repintarla: relación clave-valor pura, O(1).
enum	Genero, ColorPelo	Conjunto cerrado. Con String el compilador aceptaría "amarillo"; con enum no compila.
record	PreguntaFiltro, Suposicion, ResultadoTurno	Datos inmutables sin comportamiento: dan constructor, getters, equals y toString gratis.

Los Personaje son **inmutables** (todos los campos final) a propósito: como la máquina arma y descarta sublistas todo el tiempo, si algo pudiera cambiarle un atributo a un personaje ya cargado, las decisiones de turnos anteriores dejarían de ser válidas.

3. Bitácora de desarrollo

División del trabajo. Trabajé sola, así que no hubo reparto de tareas: el diseño, la implementación, las pruebas y esta documentación son míos. Lo que sigue es el orden real en que fui armando el proyecto, con los problemas que me fui encontrando y cómo los resolví.

Dónde están el código y la notación. Las capturas de código de cada estrategia están en la sección 4 (MergeSort en 4.1, binaria en 4.4, greedy en 4.5, descarte en 4.6, factibilidad en 4.8) y el Big O de cada función en la tabla de la sección 8.

Etapas 1 — Clases contenedoras de datos

Armé Personaje, los enums Genero y ColorPelo, y la jerarquía de Filtro.

Problema. Mi primera idea era que los personajes rapados no tuvieran color de pelo, porque parecía más realista. Al hacer la cuenta vi que eso bajaba el espacio de combinaciones de 24 a 16, y con 23 personajes iban a quedar **personajes idénticos**, imposibles de distinguir con ninguna pregunta.

Solución. Les asigné color igual, interpretado como el de las cejas o la barba, y agregué verificarUnicidad(), que lanza una excepción al arrancar si alguna vez se carga un duplicado. Prefiero que el programa reviente al iniciar antes que enterarme del problema en medio de la defensa.

Etapas 2 — Criterio de búsqueda de la máquina

Implementé la reducción de candidatos con los nombres del esquema de la cátedra (esCasoBase, descomponer, combinar) y la elección de filtro con criterio minimax.

Problema. La recursión de Divide y Conquista no aparecía en ningún método que se llamara a sí mismo, porque en el juego la reducción ocurre **a lo largo de los turnos**, y me preocupaba que no se viera como D&C. **Solución:** escribí además BuscadorRecurso, el mismo algoritmo con la recursión explícita. Las dos versiones conviven: la iterativa la usa Swing, la recursiva el modo máquina contra máquina.

Etapa 3 — Capa de servicios

CatalogoPersonajes con MergeSort e inserción binaria, OrdenadorPersonajes con los tres algoritmos de la comparación, y las dos máquinas.

Etapa 4 — Subsistemas: el motor

Partida, Oraculo y RegistroRazonamiento.

Problema. Al principio Partida tenía un bucle `while` adentro. Con la consola funcionaba, pero al armar Swing la ventana se congelaba: el hilo que dibuja la interfaz quedaba atrapado esperando input. **Solución:** la convertí en una máquina de estados que expone “a quién le toca”, “aplicó esta jugada” y “¿terminó?”, y la vista decide cuándo pedir la próxima jugada.

Etapa 5 — La interfaz de usuario

Diseñé el formulario con el Swing UI Designer de IntelliJ, definiendo componentes y nombres.

Problema. El código que genera el diseñador usa `GridLayoutManager` y `GridConstraints`, de `forms_rt.jar`, una **librería interna de IntelliJ**: sin ella el proyecto no compilaba, y aun con la dependencia bien configurada seguía fallando. Encima ese layout reparte el espacio en celdas fijas, y con 23 tarjetas más un panel que crece **los botones de pregunta quedaban fuera de la pantalla**.

Solución. Escribí `reorganizarLayout()`, que rearma la ventana con `BorderLayout` y `JScrollPane`. Ahí me di cuenta de algo: su primera instrucción es `panelPrincipal.removeAll()`, o sea que **la distribución del diseñador se descartaba tres líneas después**. Estaba peleando por una librería que construía algo que el programa tiraba. Reemplacé el método generado por nueve líneas que instancian los componentes y pasó a compilar en cualquier máquina; el `.form` queda como registro del diseño.

Otro problema. El modo máquina contra máquina necesitaba una pausa entre turnos para poder leer el razonamiento, y con `Thread.sleep` la ventana se congelaba por el mismo motivo de la etapa 4. Lo resolví con `javax.swing.Timer`, que dispara sus eventos en el hilo de Swing sin bloquearlo.

Etapa 6 — Medición

No quería afirmar “greedy es mejor” sin demostrarlo, así que programé `SimulacionEstrategias` (las 23 partidas con tres estrategias) y `BenchmarkOrdenamiento`. De ahí salieron los dos resultados inesperados de 5.1 y 6.2.

Etapa 7 — Revisión de SOLID

Repasé el código buscando dónde no cumplía y corregí varias cosas (sección 9).

Problema. Al extraer `TableroPersonajes` primero la hice heredando de `JPanel`, y eso rompió la vinculación del `.form`. Ahí entendí que la herencia no correspondía: esa clase no necesita **ser** un panel, necesita **dibujar en uno**. **Solución:** composición, recibe el panel por constructor.

Herramientas

IntelliJ IDEA 2026.2 · JDK 21 (Microsoft OpenJDK), language level 21 · Swing UI Designer · Git y GitHub · Claude (Anthropic) como asistente de IA.

Uso de inteligencia artificial

Usé **Claude (Anthropic)** durante todo el desarrollo, y lo declaro con detalle porque la consigna lo pide.

Cómo lo usé, con casos concretos:

- **Discutir el diseño.** Le planteé el enunciado y discutimos dónde correspondía cada patrón. De esa conversación salió la idea que ordenó todo el trabajo: que en cada turno hay **dos decisiones distintas** (qué preguntar y cómo achicar) y que cada una se resuelve con un patrón diferente. Antes de eso los tenía mezclados y no podía explicar dónde estaba cada uno.
- **Revisar el diseño del catálogo.** Contrastar mi primera idea sobre los personajes rapados, de donde salió el rediseño con 23 combinaciones únicas que cuento en la etapa 1.
- **Desterrar problemas de entorno.** Los de `forms_rt.jar`, el JDK mal configurado y el layout que dejaba los botones fuera de pantalla los resolví consultando: le describía el error y probábamos alternativas hasta dar con la causa.
- **Escribir código.** Buena parte del código lo escribí asistida, revisando cada cambio y compilando después de cada uno. Los nombres de métodos siguen el esquema de la cátedra por decisión explícita, para que la trazabilidad se vea.
- **Redactar esta documentación,** sobre la base de las decisiones que fuimos tomando.

4. Justificación algorítmica de Divide y Conquista

4.1 Ordenamiento inicial: MergeSort

Los 23 se declaran juntos y llegan agrupados solo por género. Como tengo **el conjunto completo de entrada**, se puede partir a la mitad, que es lo que MergeSort necesita.

```
43 @ public static List<Personaje> mergeSort(List<Personaje> lista, @ usages 2 cbaptista26
44     Comparator<Personaje> criterio) {
45
46     // CasoBase(x): una lista de 0 o 1 elemento ya esta ordenada.
47     if (lista.size() <= 1) {
48         return new ArrayList<>(lista);
49     }
50
51     // descomponer(x): dos mitades.
52     int medio = lista.size() / 2;
53     List<Personaje> izquierda = mergeSort(new ArrayList<>(lista.subList(0, medio)), criterio);
54     List<Personaje> derecha = mergeSort(new ArrayList<>(lista.subList(medio, lista.size())), criterio);
55
56     // combinar: mezclar las dos mitades ya ordenadas.
57     return mezclar(izquierda, derecha, criterio);
58 }
```

`OrdenadorPersonajes.mergeSort()`. Los comentarios marcan el esquema de la cátedra: el caso base, la descomposición en dos mitades y la combinación. Las dos flechas circulares del margen son IntelliJ señalando las llamadas recursivas.

Recurrencia: $T(n) = 2T(n/2) + \theta(n)$, porque el merge recorre los n elementos. Caso de división con $a = 2$, $b = 2$, $k = 1$. Como $a = b^k$, queda $\Theta(n \log n)$.

Aclaración honesta: con 23 personajes el juego andaría igual sin ordenar. Ordeno porque es la forma de aplicar la estrategia y porque el orden habilita la búsqueda binaria del alta individual.

4.2 MergeSort o QuickSort

	MergeSort	QuickSort
Promedio	$O(n \log n)$	$O(n \log n)$
Peor caso	$O(n \log n)$ garantizado	$O(n^2)$ con mal pivot
Espacial	$O(n)$, arrays auxiliares	$O(\log n)$, in-place
Estable	Sí	No

Elegí **MergeSort** por dos motivos. El primero: Big O se evalúa sobre el peor caso, y MergeSort garantiza $O(n \log n)$ sin importar cómo vengan los datos. El segundo es el que más me convenció: el peor caso de QuickSort se da con vectores **ya parcialmente ordenados** y mala elección de pivot, y mis personajes llegan agrupados por género, o sea parcialmente ordenados. Elegir QuickSort sería elegir justo el algoritmo cuyo caso patológico coincide con mi entrada. Su punto débil es el espacial: crea listas nuevas en cada nivel, $O(n)$ contra $O(\log n)$. Con 23 es irrelevante, pero **sería el factor limitante si el juego escalara a miles de personajes**.

4.3 Criterio de ordenamiento

Ordeno por **género** → **color de pelo** → **calvicie** → **anteojos**, encadenando comparadores con `Comparator.comparing(...).thenComparing(...)`.

Si ordenara por id la lista quedaría ordenada sola y el ordenamiento no haría nada; por atributos obliga a un reacomodamiento real. Lo importante es que define un **orden total**: como no hay dos personajes con la misma combinación, nunca devuelve 0 para dos distintos, así que cada uno tiene posición única. Eso es lo que habilita la búsqueda binaria sin ambigüedad: el criterio de orden, la estructura de datos y el algoritmo de búsqueda **se eligieron juntos**.

4.4 Construcción incremental de la lista ordenada

Esta es la parte que cumple una regla textual del enunciado: *“es la máquina quien debe disponerlos en una lista ordenada de forma autoincremental según se agregan los personajes”*. No ordeno una sola vez al final: **cada uno de los 23 se inserta en su lugar en el momento de agregarse**, con id autoincremental, y la lista está ordenada después de cada alta. Lo mismo vale si después se agrega un personaje suelto: no hay que reordenar nada.

Para ubicar dónde va cada alta uso búsqueda binaria recursiva:

```
154 private int buscarPosicion(Personaje nuevo, int ini, int fin, int profundidad) { 3 usages 3baptista26
155
156     if (ini > fin) { // caso base
157         trazaDeCarga.add(String.format(" %-20s -> %d comparacion(es), va a la posicion %d",
158             nuevo.getNombre(), profundidad, ini));
159         return ini;
160     }
161
162     int medio = (ini + fin) / 2;
163
164     if (Personaje.POR_ATRIBUTOS.compare(nuevo, ordenados.get(medio)) < 0) {
165         return buscarPosicion(nuevo, ini, fin: medio - 1, profundidad: profundidad + 1);
166     } else {
167         return buscarPosicion(nuevo, ini: medio + 1, fin, profundidad: profundidad + 1);
168     }
169 }
170
```

`CatalogoPersonajes.buscarPosicion()`. El caso base es el rango vacío, la descomposición es quedarse con media lista, y el parámetro profundidad va contando los niveles de recursión para poder mostrar la traza después.

$T(n) = T(n/2) + c \rightarrow a=1, b=2, k=0$. Como $a = b^k$, queda $O(\log n)$. Lo verifiqué contando comparaciones por alta: el primero necesita 0 y el vigésimo tercero 5. Como $\log_2(23) = 4,5$, es exactamente lo esperado; con búsqueda lineal el último habría necesitado 22.

Razonamiento de la máquina		
Aprendiz Cassandra	-> 2 comparacion(es), va a la posicion 2	
Aprendiz Delphine	-> 2 comparacion(es), va a la posicion 1	
Aprendiz Elowen	-> 3 comparacion(es), va a la posicion 3	
Aprendiz Faelynn	-> 3 comparacion(es), va a la posicion 5	
Profesora Griselda	-> 3 comparacion(es), va a la posicion 2	
Profesora Hesper	-> 3 comparacion(es), va a la posicion 5	
Profesora Ivessa	-> 4 comparacion(es), va a la posicion 8	
Profesora Jezra	-> 4 comparacion(es), va a la posicion 3	
Profesora Kaelith	-> 4 comparacion(es), va a la posicion 7	
Aprendiz Aldric	-> 4 comparacion(es), va a la posicion 11	
Aprendiz Baltasar	-> 4 comparacion(es), va a la posicion 12	
Aprendiz Ciro	-> 4 comparacion(es), va a la posicion 13	
Aprendiz Draven	-> 4 comparacion(es), va a la posicion 12	
Aprendiz Ewald	-> 4 comparacion(es), va a la posicion 14	
Aprendiz Fenwick	-> 5 comparacion(es), va a la posicion 16	
Profesor Gideon	-> 4 comparacion(es), va a la posicion 13	
Profesor Hadrian	-> 4 comparacion(es), va a la posicion 16	
Profesor Ivor	-> 5 comparacion(es), va a la posicion 19	
Profesor Joran	-> 5 comparacion(es), va a la posicion 14	
Profesor Lysander	-> 5 comparacion(es), va a la posicion 18	
Profesor Magnus	-> 5 comparacion(es), va a la posicion 22	

Modo 4: ninguna alta pasa de 5 comparaciones, aunque la lista ya tenga 22 elementos.

Verificación cruzada: el programa comprueba al arrancar que ordenar con MergeSort y construir insertando de a uno den **la misma lista**. Esto funciona como prueba porque `POR_ATRIBUTOS` es un orden total sin empates: para estos 23 personajes existe **una sola** lista ordenada posible, así que si dos implementaciones distintas llegan a la misma, las dos están bien respecto de ese criterio.

4.5 Selección del filtro y función de evaluación

La máquina decide qué preguntar con criterio **minimax**: elige el filtro cuyo peor caso deje menos candidatos. Evaluación real del primer turno:

Pregunta	Parte	Peor caso
¿Es un hombre?	12 / 11	12
¿Es calvo?	11 / 12	12

Pregunta	Parte	Peor caso
¿Usa lentes?	11 / 12	12
¿Pelo colorado?	8 / 15	15
¿Pelo negro?	8 / 15	15
¿Pelo amarillo?	7 / 16	16

Si eligiera pensando en el caso favorable preguntaría “¿Tiene el pelo amarillo?” esperando un sí que deja 7; pero si le dicen que no quedan 16, **peor que antes de preguntar**. Como no controla la respuesta, lo único que puede controlar es qué tan mal le puede ir.

La métrica es `Particion.peorCaso()`, que devuelve `Math.max(cumplen, noCumplen)`. El **descarte garantizado** es `total - peorCaso`: en el turno 1, “¿Es un hombre?” descarta al menos 11 pase lo que pase.

La **primera pregunta no cambia entre partidas** (con 23 candidatos la evaluación da siempre lo mismo), pero **desde el turno 2 sí**, porque el conjunto ya depende de la respuesta del rival. **Ante empates me quedo con el primero de la lista**, no con uno al azar, para que la máquina sea determinista y las mediciones de la sección 6 sean comparables entre sí. Si la lista de factibles quedara vacía arriesga con `candidatos.get(0)`, $O(1)$: en mi catálogo no llega a pasar, pero el código lo contempla.

```

59 @Override usage &cbaptista26
60 @
61 protected Filtro elegirFiltro(List<Filtro> factibles) {
62
63     ultimaEvaluacion = new ArrayList<>();
64     EvaluacionFiltro mejor = null;
65
66     for (Filtro filtro : factibles) {
67         EvaluacionFiltro e = new EvaluacionFiltro(filtro, descomponer(candidatos, filtro));
68         ultimaEvaluacion.add(e);
69
70         // Si dos filtros empatan me quedo con el primero de la lista. Lo
71         // decidí así para que la máquina sea determinista: con el mismo
72         // personaje secreto siempre juega igual, y eso me deja reproducir
73         // una partida para mostrarla. Si desempata al azar, cada corrida
74         // daría distinto y no podría compararlos.
75         if (mejor == null || e.peorCaso() < mejor.peorCaso()) mejor = e;
76     }
77
78     explicarDecision(mejor);
79     return mejor.getFiltro();
80 }

```

`MaquinaGreedy.elegirFiltro()`. El ciclo evalúa cada filtro factible y se queda con el de menor peor caso: es la función de selección del esquema greedy, y el $O(f \cdot n)$ de la tabla de la sección 8 sale de este `for` con el recorrido de candidatos adentro de `descomponer`.

4.6 Descarte de candidatos y árbol de decisión

El esquema dice	En mi juego es	Método
x	los personajes que todavía pueden ser	campo candidatos
<code>CasoBase(x)</code>	queda uno solo	<code>esCasoBase()</code>
<code>SoluciónDirecta(x)</code>	ese es el personaje	<code>solucionDirecta()</code>
<code>descomponer(x)</code>	separar en cumple / no cumple	<code>descomponer()</code>
combinar	seguir con el grupo de la respuesta real	<code>recibirRespuesta()</code>

```

93 /** descomponer(x): parte los candidatos en dos subconjuntos disjuntos. Theta(n). */
94 @
95 public static Particion descomponer(List<Personaje> candidatos, Filtro filtro) { 6 usages &cbaptista26
96
97     List<Personaje> cumplen = new ArrayList<>();
98     List<Personaje> noCumplen = new ArrayList<>();
99
100     for (Personaje p : candidatos) {
101         if (filtro.evaluar(p)) cumplen.add(p);
102         else noCumplen.add(p);
103     }
104     return new Particion(cumplen, noCumplen);
105 }

```

`Jugador.descomponer()`. Es el corazón del trabajo: una sola pasada sobre los candidatos que los parte en dos subconjuntos disjuntos. Está en la clase padre porque el tablero del humano se achica con el mismo algoritmo que el de la máquina.

El mecanismo. Recorro los candidatos una sola vez y armo dos listas; con la respuesta me quedo con una y **descarto la otra entera**. Costo $O(n)$, lineal e imposible de bajar: hay que mirar a cada personaje al menos una vez.

Creo dos listas nuevas en vez de ir borrando de la existente, y no es por costo: `removeIf()` también haría una sola pasada $O(n)$. Es porque necesito **los dos lados a la vez**. La máquina tiene que poder *evaluar* un filtro —saber cuánto cortaría— sin tocar sus candidatos, y para eso `peorCaso()` necesita el tamaño de los dos grupos. Si borra in situ, evaluar un filtro destruiría el estado y no podría comparar seis.

Una diferencia con el esquema genérico: el esquema resuelve *todos* los subproblemas y después combina; acá resuelvo **uno solo**, porque la respuesta me dice en cuál de los dos grupos está el personaje. Es la misma simplificación de la búsqueda binaria.

Dónde está la recursión. Lo que más me costó entender: no hay ningún método que se llame a sí mismo, está repartida **a lo largo de los turnos**.

```

descomponer(23) con "¿Es un hombre?" → [12 | 11] SI → quedan 12
descomponer(12) con "¿Es calvo?" → [6 | 6] SI → quedan 6
descomponer(6) con "¿Usa lentes?" → [3 | 3] NO → quedan 3
descomponer(3) con "¿Pelo colorado?" → [1 | 2] NO → quedan 2
descomponer(2) con "¿Pelo negro?" → [1 | 1] SI → queda 1
CasoBase → SoluciónDirecta

```

23 → 12 → 6 → 3 → 2 → 1, la misma forma que el 100 → 50 → 25 → 12 → 6 → 3 → 1 del ejemplo de clase.

El árbol de decisión. Una partida es un recorrido de la raíz a una hoja de un árbol binario: los nodos son preguntas, las ramas las respuestas, las hojas los personajes. Con 23 hojas la altura mínima es 5, y **el árbol no se construye en memoria**: se recorre de forma perezosa con los candidatos que quedan.

La **greedy** arma un árbol equilibrado, cercano al mínimo. La **secuencial** pregunta en orden fijo empezando por los colores, los filtros más desparejos, así que sus primeros niveles quedan desbalanceados y el árbol termina más alto. Pero **no juega al azar**: no hay ningún Random en su lógica.

4.7 Criterio de parada y suposición final

Son dos cosas distintas y las separo.

(a) Fin del turno. La función de corte es `esCasoBase()`. Si quedan dos o más candidatos, la máquina elige un filtro y pregunta: ahí termina su turno, porque la respuesta no depende de ella. Si queda uno solo, pasa a la suposición. El turno termina siempre después de **una** acción, nunca las dos.

(b) Fin del juego. Termina cuando alguien arriesga. Si acierta gana; **si se equivoca pierde**. Elegí esa regla porque es la que le da sentido a arriesgar: si equivocarse no costara nada, lo óptimo sería adivinar desde el turno 1. Con esa regla MáquinaGreedy **solo arriesga con un único candidato** —salvo el caso degenerado de 4.5, inalcanzable en este catálogo—, o sea con probabilidad 1 de acertar, y acá se conecta con el diseño del catálogo: **cuando llega al caso base, el candidato que queda ES el personaje secreto**. Eso funciona porque las 23 combinaciones son únicas; con repetidos el caso base dejaría dos o tres empatados y la suposición sería una moneda al aire.

4.8 Dependencia lógica entre filtros

Los seis filtros **no son todos independientes**. Género, calvicie y anteojos sí, pero los tres de color están ligados: si el pelo es colorado, no es negro ni amarillo.

Consecuencia: después de “¿colorado? NO” y “¿negro? NO”, el filtro “¿amarillo?” **ya está determinado** y preguntarlo gasta un turno sin descartar a nadie. Lo resuelve la función de factibilidad:

```
47     protected List<Filtro> filtrosFactibles() { 1 usage  2 cbaptista26
48         List<Filtro> factibles = new ArrayList<>();
49
50         for (Filtro f : CatalogoFiltros.todos()) {
51             if (filtrosUsados.contains(f.getClave())) continue;
52             if (aplicaFactibilidad() && descomponer(candidatos, f).esInutil()) continue;
53             factibles.add(f);
54         }
55         return factibles;
56     }
57 }
```

`JugadorMáquina.filtrosFactibles()`. Las dos guardas del ciclo son la función de factibilidad: la primera saltea los filtros ya preguntados, la segunda los que no partirían nada. Es lo que la medición de la sección 6.2 valúa en ~1,35 turnos.

No lo resolví con reglas escritas a mano del tipo “si ya preguntaste dos colores no preguntes el tercero”, sino **midiendo la partición real** sobre los candidatos actuales. Así funciona para cualquier dependencia entre atributos, no solo la de los colores.

Esta dependencia no se filtra al diseño de clases: los seis filtros son subclases hermanas y ninguna conoce a las otras. La relación aparece solo en ejecución, al medir particiones.

4.9 Cómo impido que la máquina vea el personaje del humano

La otra regla textual del enunciado: “*la máquina no sabe, no puede acceder directamente a la variable del personaje elegido por el jugador humano*”.

No lo resolví con disciplina ni con un comentario pidiendo que nadie lo mire. **La máquina nunca recibe un Personaje**: recibe un `Oraculo`, una interfaz con dos operaciones, preguntar por un filtro y arriesgar un nombre. El personaje secreto vive en un campo `private final` de `OraculoPersonaje` **sin getter**, así que no es que la máquina no lo hace: no compilaría si lo intentara. El único método que lo devuelve es `revelarAlFinalizar()`, y `Partida.getSecretoDe()` lanza excepción si la partida sigue en curso. La ventaja de resolverlo con el tipo y no con una convención es que la restricción la verifica el compilador, no yo leyendo el código.

4.10 Precondiciones y validaciones

Validación	Dónde	Qué previene
23 personajes con combinaciones únicas	<code>VerificadorCatalogo.verificarUnicidad()</code>	Candidatos empatados imposibles de separar
MergeSort y binaria dan la misma lista	<code>verificarQueCoincidan()</code>	Que uno de los dos algoritmos esté mal
No se revela el secreto en curso	<code>Partida.getSecretoDe()</code>	Que una vista haga trampa
No se juega una partida terminada	<code>Partida.aplicarJugada()</code>	Estados inconsistentes

Las dos primeras corren al arrancar y lanzan excepción si fallan. Además, los constructores de `Jugador` y `OraculoPersonaje` rechazan una lista de candidatos vacía y un secreto nulo.

4.11 Los pensamientos de la máquina, logueados

La máquina no decide en silencio: **imprime su razonamiento completo en la consola y en el panel de la interfaz gráfica**.

Razonamiento de la máquina

```
[SECUENCIAL] SELECCION secuencial: "Tiene el pelo amarillo?", el primero pendiente del orden fijo. No evaluo cuanto corta.
[SECUENCIAL] respuesta NO -> combinar: sigo con "no cumplen". Candidatos 8 -> 8 (descarte 0%)
>> SECUENCIAL pregunta "Tiene el pelo amarillo?" -> NO. Candidatos: 8 -> 8

--- TURNO 7 - juega GREEDY (3 candidatos) ---
[GREEDY] evaluo 3 filtros sobre 3 candidatos:
-> Tiene el pelo colorado? parte 1/2 peor caso: 2
Tiene el pelo negro? parte 1/2 peor caso: 2
Tiene el pelo amarillo? parte 1/2 peor caso: 2
[GREEDY] SELECCION greedy: "Tiene el pelo colorado?" porque su peor caso (2) es el menor. Descarta al menos 1 candidatos pase lo que pase.
[GREEDY] respuesta NO -> combinar: sigo con "no cumplen". Candidatos 3 -> 2 (descarte 33%)
>> GREEDY pregunta "Tiene el pelo colorado?" -> NO. Candidatos: 3 -> 2

--- TURNO 8 - juega SECUENCIAL (8 candidatos) ---
```

Dos turnos seguidos del modo máquina vs máquina. Arriba la secuencial gasta un turno entero (8 → 8, descarte 0%); abajo la greedy evalúa los tres filtros que le quedan y elige.

Se lee línea por línea qué patrón actúa: las de evaluo son **greedy evaluando**, la de SELECCION es **la decisión**, la de combinar es **D&C quedándose con un solo subproblema**. Funciona porque el motor **nunca llama a System.out**: le pasa los mensajes a la interfaz RegistroRazonamiento, con tres implementaciones (consola, Swing, silenciosa para las simulaciones), así que el mismo log aparece idéntico en las dos vistas sin duplicar código.

5. Tabla comparativa de tiempos

Medido con `System.nanoTime()` sobre la misma lista de 23 personajes, promediando 20.000 repeticiones por ejecución con calentamiento previo de la JVM.

Algoritmo	Complejidad	Ejec. 1	Ejec. 2	Ejec. 3	Promedio
MergeSort	$O(n \log n)$	0,0128 ms	0,0076 ms	0,0079 ms	0,0094 ms
Burbujeo	$O(n^2)$	0,0075 ms	0,0058 ms	0,0055 ms	0,0063 ms
Inserción simple	$O(n^2)$	0,0038 ms	0,0025 ms	0,0023 ms	0,0029 ms

5.1 ¿Es significativa la diferencia para $n = 23$?

No, y hay que decirlo con todas las letras: MergeSort quedó último. La inserción simple fue **más de tres veces más rápida** que el algoritmo $n \log n$.

Dos motivos: con n chico las **constantes pesan más que el orden** (MergeSort reserva memoria en cada nivel), y mis personajes llegan **agrupados por género, o sea parcialmente ordenados**, que es la mejor entrada posible para inserción, cuyo mejor caso es $O(n)$. Para comprobarlo medí lo mismo con la lista **desordenada al azar**:

Algoritmo	Tal como llegan	Desordenados
MergeSort	0,0094 ms	0,0072 ms
Burbujeo	0,0063 ms	0,0102 ms
Inserción simple	0,0029 ms	0,0054 ms

MergeSort **mejoró** al desordenar, y los dos cuadráticos empeoraron tanto que el burbujeo pasó a ser el más lento de los tres. Eso confirma que la ventaja de los cuadráticos venía del orden de entrada, no del algoritmo. MergeSort se mueve poco entre un caso y el otro, que es exactamente lo que significa tener el peor caso garantizado.

5.2 Pero eso no valida la elección

Esta tabla es una medición distinta de la anterior: usa **personajes generados al azar**, no los 23 del juego, justamente para ver qué pasa cuando n crece. Por eso el renglón de $n=23$ no coincide con la tabla de arriba y no hay que compararlos entre sí.

n	MergeSort	Burbujeo	Inserción	Burbujeo/MergeSort
23	0,0064 ms	0,0082 ms	0,0043 ms	1,3×
100	0,0427 ms	0,2164 ms	0,0591 ms	5,1×
500	0,2178 ms	5,157 ms	2,112 ms	23,7×
2.000	1,516 ms	97,775 ms	28,589 ms	64,5×
5.000	3,852 ms	514,427 ms	156,877 ms	133,5×

En comparaciones puras la proyección es todavía más agresiva: $n=1.000 \rightarrow \sim 9.966$ contra 1.000.000 (100×). Los factores medidos quedan por debajo porque la proyección cuenta **solo comparaciones** y el tiempo real incluye la memoria que reserva MergeSort, un costo constante que la teoría no ve. Eso achica la brecha sin cambiar la tendencia.

Conclusión doble. Para 23 personajes elegir uno u otro no cambia nada perceptible, y si el criterio fuera la velocidad medida hoy ganaría la inserción simple. Elijo MergeSort porque su $O(n \log n)$ es una **garantía, no un promedio**: los cuadráticos son rápidos solo si tienen suerte con la entrada, y acá la tuvieron.

6. Justificación de la estrategia Greedy

6.1 Los cinco elementos

Elemento	En mi juego	Dónde
Conjunto de candidatos	los 6 filtros del catálogo	<code>CatalogoFiltros.todos()</code>
Función de selección	el de menor peor caso	<code>MaquinaGreedy.elegirFiltro()</code>
Función de factibilidad	no usado y que separe de verdad	<code>filtrosFactibles()</code>
Función de solución	queda un solo personaje	<code>esCasoBase()</code>

Elemento	En mi juego	Dónde
Objetivo	minimizar la cantidad de preguntas	

Por qué es eficiente. Es el mismo razonamiento de la búsqueda binaria: cada pregunta bien elegida descarta la mitad, así que con 23 personajes hacen falta unas 5. Una estrategia lineal ("¿sos Aurelia?", "¿sos Briseida?") necesitaría hasta 23. La evaluación cuesta $O(f \cdot n)$ por turno: con $f=6$ y $n \leq 23$, a lo sumo 138 comparaciones a cambio de pasar de 23 turnos a 6. Es greedy de verdad porque **decide mirando solo el turno actual**: no simula los siguientes ni reconsidera preguntas hechas. Es "corto de vista" a propósito, igual que el cambio de monedas visto en clase.

6.2 Medición: qué aporta cada elemento

SimulacionEstrategias juega **las 23 partidas posibles** con tres estrategias sobre el mismo motor.

Aclaración de unidad: **un turno es una pregunta, y la suposición final consume un turno más**, así que 6 turnos son 5 preguntas más el acierto.

Estrategia	Peor caso	Promedio	Aciertos
Greedy completo	6 turnos	5,61	23/23
Secuencial con factibilidad	6 turnos	5,61	23/23
Secuencial pura	7 turnos	6,96	23/23

Elemento	Mejora del promedio
Función de factibilidad sola	-1,35 turnos
Función de selección minimax	0,00 turnos

El resultado no fue el que esperaba. Lo que realmente mejora el rendimiento no es elegir la mejor pregunta, sino descartar las que ya tienen respuesta determinada: en la captura de la sección 4.11 se ve a la secuencial gastando un turno entero sin sacar a nadie del tablero.

```

Razonamiento de la maquina
Greedy (seleccion + factibilidad) 6 5 5,61 23/23
Secuencial CON factibilidad 6 5 5,61 23/23
Secuencial pura (sin greedy) 7 6 6,96 23/23

QUE APORTA CADA ELEMENTO DEL ESQUEMA GREEDY
Factibilidad sola : -1.35 turnos de promedio (7 -> 6 en el peor caso)
Selección minimax : +0.00 turnos de promedio (6 -> 6 en el peor caso)
Greedy completo : -1.35 turnos de promedio (7 -> 6 en el peor caso)

=====
CONTRASTE CONTRA LA COTA TEORICA
=====
Personajes (n) : 23
Cota inferior teorica techo(log2 n) : 5 preguntas
Preguntas de greedy en el peor caso : 5
Turnos totales (preguntas + 1 suposición): 6

Greedy alcanza la cota inferior: ninguna estrategia puede
identificar a uno entre 23 personajes con menos preguntas,
porque cada pregunta binaria aporta como máximo 1 bit.

Fuerza bruta (probar uno por uno) : hasta 23 intentos, O(n)

```

Modo 3: las 23 partidas posibles con cada estrategia, y el contraste contra la cota teórica.

¿Por qué la selección minimax no aporta nada? Porque en mi catálogo el techo ya está fijado por la estructura, no por el orden de las preguntas. Identificar a un personaje es determinar sus 4 atributos, y eso cuesta **como mucho 5 preguntas en cualquier orden que descarte las inútiles**: una de género, una de calvicie, una de lentes y a lo sumo dos de los tres colores. Elegir mejor cambia el camino, no la cantidad. Por eso la secuencial con factibilidad empata con la greedy aunque arranque por "¿pelo colorado?", que parte 8/15 y por minimax es claramente peor.

La mantengo igual porque garantiza no elegir nunca un filtro malo, y porque con un catálogo desbalanceado —donde el techo no estuviera fijado— sí haría diferencia. Lo que la medición me mostró es que **la ventaja de un algoritmo voraz depende de cómo estén armados los datos, no solo del algoritmo**.

6.3 ¿Hay un escenario donde mi Greedy no sea óptimo?

Para mi catálogo, no, y se demuestra con un argumento de información:

Cada pregunta de sí/no aporta como máximo 1 bit. Para distinguir entre 23 personajes hacen falta al menos $\log_2(23)$ redondeado para arriba = 5 preguntas, haga lo que haga cualquier algoritmo. Mi greedy resuelve en 5 preguntas en el peor caso (los 6 turnos de la tabla son esas 5 más la suposición). Como el mínimo teórico y el resultado coinciden, **ninguna estrategia puede hacerlo mejor**.

Y también es óptimo **en promedio**, no solo en el peor caso. Un árbol binario con 23 hojas tiene una longitud de camino mínima de $23 \cdot 5 - (32 - 23) = 106$, o sea $106/23 = 4,61$ preguntas de promedio. Mi greedy mide 5,61 turnos, que son exactamente 4,61 preguntas. Da el mínimo teórico también acá.

Pero eso vale para este catálogo, no en general, y es la lección del cambio de monedas: con el sistema decimal greedy da el óptimo, con el británico previo a 1971 el mismo algoritmo da 4 monedas cuando el óptimo son 3. No cambió el algoritmo, cambiaron los datos.

Tres escenarios donde el mío fallaría: con **atributos correlacionados**, la pregunta que más descarta ahora podría dejar un subconjunto imposible de partir bien después, y greedy no mira el futuro (ahí ganaría el árbol óptimo por programación dinámica); con una **distribución desbalanceada**, donde si todos los filtros partieran muy desparejo el peor caso se degradaría hacia $O(n)$; y **los empates**, donde elegir el primero es arbitrario y greedy nunca reconsidera.

Una precisión para no contradecirme con la sección 4.8: mis cuatro **atributos** sí son independientes entre sí, pero mis seis **filtros** no, porque los tres de color son mutuamente excluyentes. Esa dependencia existe y la neutraliza la función de factibilidad. En resumen: **mi greedy es óptimo porque mi catálogo es uniforme y sus atributos independientes**, no porque el algoritmo lo sea por naturaleza.

7. Algoritmos vistos en clase que NO utilicé

Algoritmo	Por qué no aplica
QuickSort	Su peor caso $\Theta(n^2)$ se da con vectores parcialmente ordenados, que es cómo llegan mis personajes (4.2).
Dijkstra	Requiere grafo dirigido con costos y un origen. Mi juego no modela distancias entre personajes: el descarte es por atributos booleanos, no por peso de aristas.
Prim / Kruskal	Resuelven árboles de recubrimiento mínimo sobre grafos con costos. No existe el problema de conectar los personajes al mínimo costo: son elementos independientes, no vértices con aristas.
Código de Huffman	Es compresión por frecuencia de símbolos; acá no hay codificación. Comparte la idea del árbol binario, pero Huffman lo construye de abajo hacia arriba combinando frecuencias y el mío se recorre de arriba hacia abajo eligiendo preguntas.
Fibonacci recursivo	$O(2^n)$, en realidad $\Theta(\varphi^n)$ con $\varphi \approx 1,618$. Acá no hay que resolver las dos ramas: la respuesta dice cuál seguir y la otra se descarta entera.
Programación dinámica	Dentro de una partida no hay nada que memorizar: se recorre una sola rama del árbol y nunca se vuelve a un subconjunto ya visto. El solapamiento aparece solo si quisiera precomputar el árbol óptimo (a {masculino, calvo} se llega por género $\rightarrow$ calvicie y por calvicie $\rightarrow$ género, es el mismo estado), y ahí sí correspondería PD sobre subconjuntos. No la apliqué porque greedy ya alcanza la cota inferior (6.3). $6! = 720$ órdenes, tratable, pero no aporta: greedy ya alcanza la cota inferior (6.3).
Fuerza bruta sobre el orden de preguntas	Lo evalué: partir la lista de filtros, buscar el mínimo en cada mitad, combinar. Lo descarté porque hallar el mínimo obliga a mirar todos los elementos igual: no baja el orden (sigue $\Theta(f \cdot n)$, porque hay que evaluar la partición de cada filtro sobre los n candidatos) y solo agrega llamadas a la pila.
D&C para elegir el filtro	Implementados, pero no se usan en el juego : solo para la tabla de la sección 5.
Burbujeo e inserción simple	

8. Notación Big O para la aplicación

Uso dos variables: **n** = personajes (23) y **f** = filtros (6). Las separo porque son magnitudes independientes: podría agregar filtros sin agregar personajes.

Función / método	Temporal	Espacial	Justificación
OrdenadorPersonajes.mergeSort()	$\Theta(n \log n)$	$\Theta(n)$	$T(n) = 2T(n/2) + \Theta(n)$, $a=2$, $b=2$, $k=1$
OrdenadorPersonajes.burbujeo()	$O(n^2)$	$\Theta(n)$	Dos ciclos anidados. No se usa en el juego
OrdenadorPersonajes.insercionSimple()	$O(n^2)$	$\Theta(n)$	Mejor caso $O(n)$ si ya viene ordenada. No se usa en el juego
CatalogoPersonajes.buscarPosicion()	$\Theta(\log n)$	$\Theta(\log n)$	Binaria: $T(n) = T(n/2) + c$, $a=1$, $b=2$, $k=0$. El espacial es la pila de recursión
CatalogoPersonajes.insertarOrdenado()	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$ para ubicar + $\Theta(n)$ de desplazamiento
CatalogoPersonajes.cargarPersonajes()	$\Theta(n^2)$	$\Theta(n)$	n altas de $\Theta(n)$ desplazamiento cada una
VerificadorCatalogo.verificarUnicidad()	$\Theta(n)$	$\Theta(n)$	Una pasada con HashSet, add() en $O(1)$
Jugador.descomponer()	$\Theta(n)$	$\Theta(n)$	Un recorrido de los candidatos
Jugador.recibirRespuesta()	$\Theta(n)$	$\Theta(n)$	Llama a descomponer y reemplaza la lista
MaquinaGreedy.elegirFiltro()	$\Theta(f \cdot n)$	$\Theta(f \cdot n)$	Ciclo anidado : por cada filtro, recorre candidatos. Guarda las f particiones para poder mostrarlas
JugadorMaquina.filtrosFactibles()	$\Theta(f \cdot n)$	$\Theta(f + n)$	Ciclo anidado : evalúa la partición de cada filtro y la descarta
BuscadorRecursivo.resolver()	$\Theta(f \cdot n)$	$\Theta(n \log n)$	$T(n) = T(n/2) + \Theta(f \cdot n)$, $a=1$, $b=2$, $k=1$. El espacial es la pila por las sublistas
Partida.aplicarJugada()	$\Theta(n)$	$\Theta(n)$	Delega en recibirRespuesta, que crea listas nuevas
TableroPersonajes.pintar()	$\Theta(n)$	$\Theta(n)$	HashSet de ids vivos + una pasada por tarjetas
CatalogoPersonajes.buscarPorNombre()	$O(n)$	$O(1)$	Lineal; solo al elegir personaje
Una partida completa	$\Theta(f \cdot n)$	$\Theta(n)$	Ver abajo
La aplicación	$\Theta(n^2)$	$\Theta(n)$	Dominada por la carga incremental del catálogo

Las dos funciones con **ciclos anidados** son elegirFiltro() y filtrosFactibles(): un for sobre los filtros y adentro un recorrido de candidatos. Quedan en $\Theta(f \cdot n) = 138$ operaciones por turno como máximo.

El costo de una partida completa. Uno esperaría $\Theta(f \cdot n \log n)$, pero el conjunto se achica a la mitad en cada turno, así que la suma es una serie geométrica: $f \cdot (n + n/2 + n/4 + \dots + 1) = f \cdot 2n \rightarrow \Theta(f \cdot n)$. O sea que **el primer turno cuesta tanto como todos los demás sumados**, que me pareció el resultado más lindo del análisis porque es contraintuitivo.

La notación más certera: $\Theta(n^2)$, y no el $\Theta(n \log n)$ del MergeSort. El arranque hace dos cosas: ordena el lote con MergeSort, $\Theta(n \log n)$, y además construye la lista alta por alta con insertarOrdenado(), que ubica en $\Theta(\log n)$ pero **desplaza en $\Theta(n)$** . Son n altas de $\Theta(n)$, o sea $\Theta(n^2)$: 529 desplazamientos contra las ~104 comparaciones del MergeSort. Jugar aporta $\Theta(f \cdot n)$, que con f constante es $\Theta(n)$. El término que manda es el cuadrático.

Acepto ese $\Theta(n^2)$ pudiendo no hacerlo porque **lo pide el enunciado**, que exige disponer los personajes en una lista ordenada a medida que se agregan (sección 4.4); si el objetivo fuera solo la velocidad, alcanzaría con el MergeSort del lote. Con 23 personajes los 529 desplazamientos son irrelevantes. Escalando a **1.000**, las preguntas por partida subirían solo de 5 a 10 y **jugar seguiría siendo instantáneo**, pero el arranque pasaría a 1.000.000 de desplazamientos y habría que cambiar a un árbol balanceado, donde insertar cuesta $\Theta(\log n)$. Aunque el límite real llega antes: con 1.000 personajes **el catálogo ya no podría tener combinaciones únicas**, porque 4 atributos generan solo 24.

9. Principios SOLID y acoplamiento

Principio	Cómo lo cumplo
S — Responsabilidad única	Extraje cuatro clases de otras que hacían demasiado: VerificadorCatalogo del catálogo, que ya cargaba y ordenaba, y ControladorPartida, TableroPersonajes y PaletaArcana de VentanaJuego, que armaba el layout, dibujaba las tarjetas, manejaba la partida y definía los colores. Cambiar la temática y cambiar el comportamiento de la ventana son motivos de cambio distintos.
O — Abierto/Cerrado	Es lo más fuerte del diseño. Agregar una característica nueva (por ejemplo "¿lleva sombrero?") es crear una subclase de Filtro y sumarla a CatalogoFiltros.todos(), sin tocar una línea de MáquinaGreedy ni de Jugador .
L — Sustitución de Liskov	Partida trabaja con Jugador y nunca pregunta de qué tipo es . Antes las vistas casteaban a JugadorMáquina y usaban instanceof para el humano: cuatro casteos. Agregué Jugador.decidirJugada(), que devuelve Optional<Jugada> — las máquinas devuelven su jugada, el humano devuelve vacío — y desaparecieron todos .
I — Segregación de interfaces	Oraculo tiene 2 métodos y RegistroRazonamiento tiene 1 abstracto más 1 default: quien lo implementa escribe solo registrar(), y registrarTitulo() viene resuelto. Nadie implementa métodos que no usa. Oraculo es el caso más claro: expone exactamente lo que la máquina puede hacer y nada más. Jugador llamaba a CatalogoPersonajes.getInstancia(): dependía de una clase concreta y de un singleton. Ahora recibe la lista de candidatos por constructor , así que puedo crear una máquina con un catálogo de prueba de 4 personajes y verificar el algoritmo sin tocar el real. Antes era imposible.
D — Inversión de dependencias	

Sobre el acoplamiento, lo que más me interesa señalar es que **el motor no conoce a las vistas**: la dirección de las dependencias es siempre de afuera hacia adentro, app y vista dependen de juego y jugador y nunca al revés. ControladorPartida es la pieza que lo hace posible: la ventana le pide jugadas y él habla con el motor, así que Partida no conoce a Swing. Por eso el juego funciona igual en consola y en Swing sin cambiar una línea del motor, y por eso existe RegistroSilencioso: para correr 23 simulaciones sin que el motor se entere de que nadie está mirando.

Composición en lugar de herencia. El caso de TableroPersonajes (etapa 7) me sirvió para ver la diferencia en un caso concreto en vez de en abstracto.

Lo que sigue sin ser perfecto. VentanaJuego sigue siendo la clase más grande y todavía hace tres cosas: layout, diálogos y despacho de los cuatro modos. Se podría partir más; no lo hice porque preferí dejar de tocar código que ya andaba y estaba probado.

10. Reflexión sobre el trabajo

Logros

Lo que más rescato es haber podido **medir en vez de suponer**. Las dos afirmaciones más fuertes del trabajo —que greedy llega al óptimo y que MergeSort no siempre conviene— están respaldadas por programas que corren y dan números, y las dos mediciones me dieron resultados que **no esperaba**. También rescato la separación entre motor y vistas, que me permitió agregar los cuatro modos sin tocar la lógica, y el catálogo con combinaciones únicas: una decisión chica, casi de datos, que evita que todo el análisis se caiga justo en el último turno.

Dificultades

Swing me llevó más tiempo que toda la parte algorítmica junta. Primero el JDK: IntelliJ tenía varias versiones registradas que ya no existían en mi máquina y el proyecto no compilaba, hasta que descargué el Temurin 21 desde el propio IDE. Después el plugin del diseñador de formularios, que en la versión 2026 ya no viene instalado y hay que buscarlo en el Marketplace. Y cuando por fin compiló, la ventana abría con los botones de pregunta fuera de la pantalla. Ninguno de los tres tenía que ver con el algoritmo y los tres me frenaron.

La otra dificultad fue entender **dónde estaba la recursión** de Divide y Conquista, porque ningún método se llama a sí mismo. Lo terminé de entender escribiendo las dos versiones del mismo algoritmo, la que usa la interfaz y la recursiva literal, y viendo que hacen lo mismo.

Mejoras posibles

- **Agregar un filtro nuevo**, por ejemplo "¿lleva sombrero?". Es crear una subclase de Filtro y sumarla a la lista: no hay que tocar la máquina. Sería la prueba práctica de que el diseño quedó bien abierto.
- **Mostrar el árbol de decisión en pantalla**, con el camino de la partida marcado. Hoy el árbol existe pero solo se ve como texto en el panel.
- **Construir el árbol óptimo con programación dinámica** y compararlo contra el greedy. Con mi catálogo creo que daría lo mismo, porque greedy ya llega a las 5 preguntas, pero está sin probar.
- **Tests con JUnit**. Las verificaciones que tengo corren al arrancar el programa: funciona, pero lo normal es tenerlas aparte.

11. Bibliografía, fuentes y ayudas

Material de la cátedra. Diapositivas de Divide y Conquista (partes 1 y 2), Greedy (parte 1), Big O e Introducción a la Optimización.

Bibliografía. Cormen, Leiserson, Rivest y Stein, *Introduction to Algorithms*, 3.ª ed., capítulos 2 (MergeSort y recurrencias) y 16 (algoritmos voraces). Brassard y Bratley, *Fundamentos de Algoritmia*, capítulos de Divide y Conquista y de algoritmos voraces.

Documentación técnica. Documentación oficial de Java 21 (java.util.Comparator, java.util.List, java.util.Optional, javax.swing.Timer, record, sealed interface). Documentación de IntelliJ IDEA sobre el Swing UI Designer.

Herramientas. IntelliJ IDEA 2026.2 · JDK 21 (Microsoft OpenJDK) · Git y GitHub · Swing UI Designer.

Asistencia de inteligencia artificial. Claude (Anthropic).

Anexo — Cómo ejecutar

```
javac -d out $(find src -name "*.java")
java -cp out com.tpo.adivinaquien.vista.VentanaJuego # interfaz gráfica
java -cp out com.tpo.adivinaquien.app.JuegoConsola # menú de consola
```

Los cuatro modos están en la consola y en el desplegable de la ventana. El proyecto no depende de ninguna librería externa: compila con Java 21 y nada más.

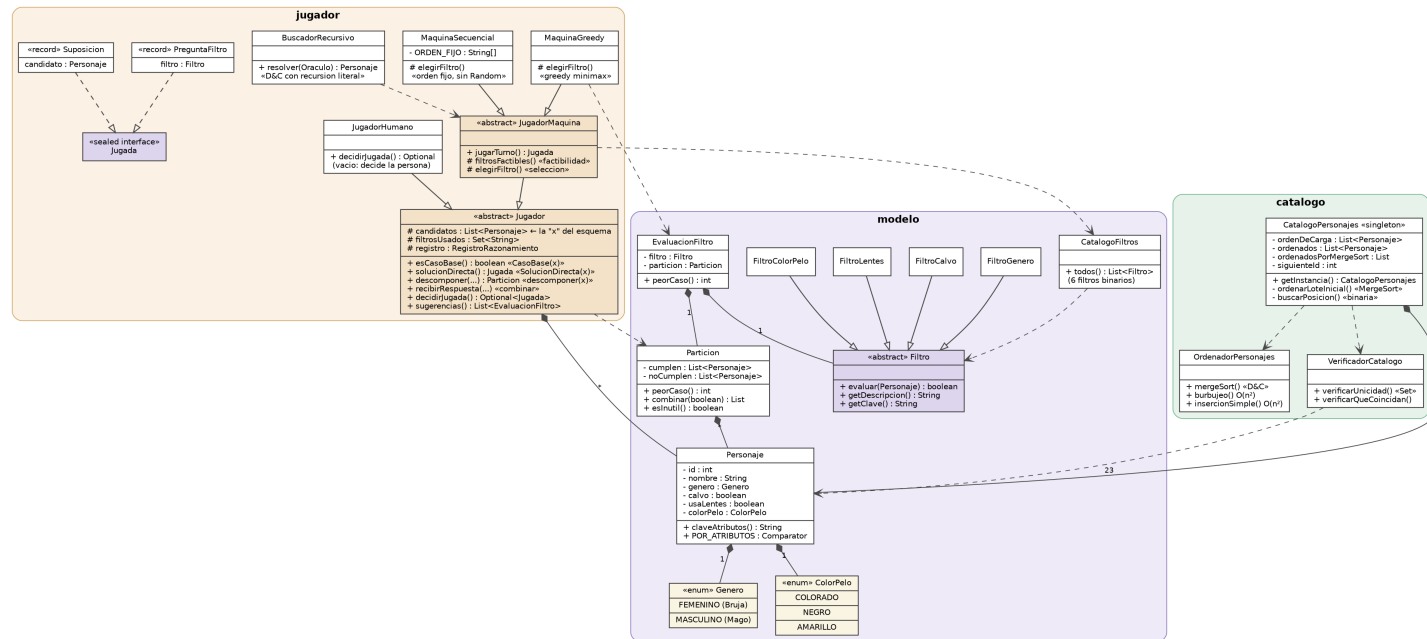


Diagrama 1 — Núcleo algorítmico (paquetes modelo, catalogo y jugador)

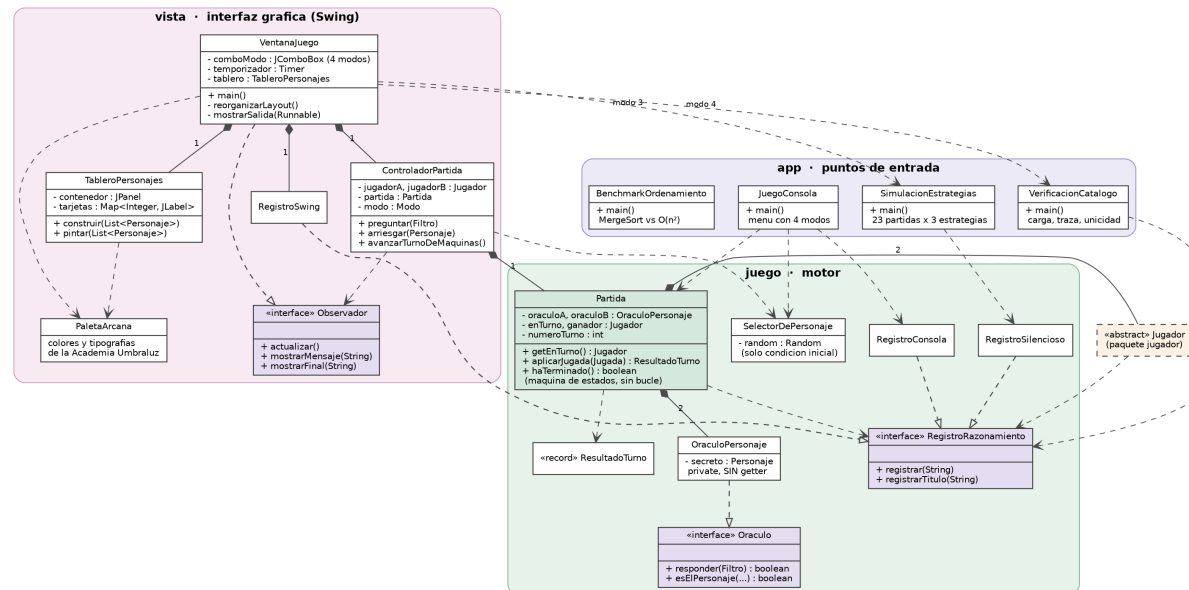


Diagrama 2 — Motor y vistas (paquetes juego, app y vista)